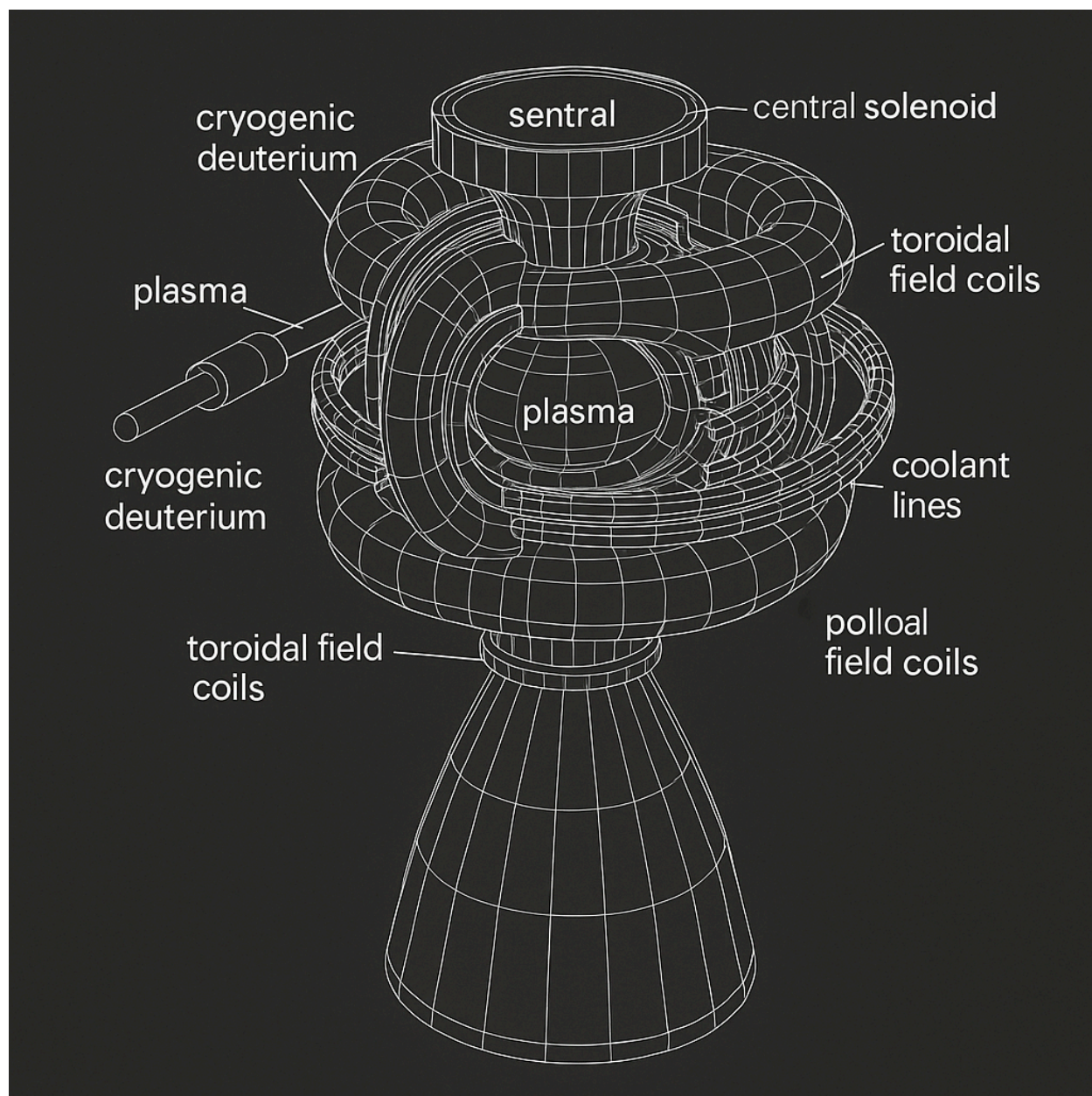
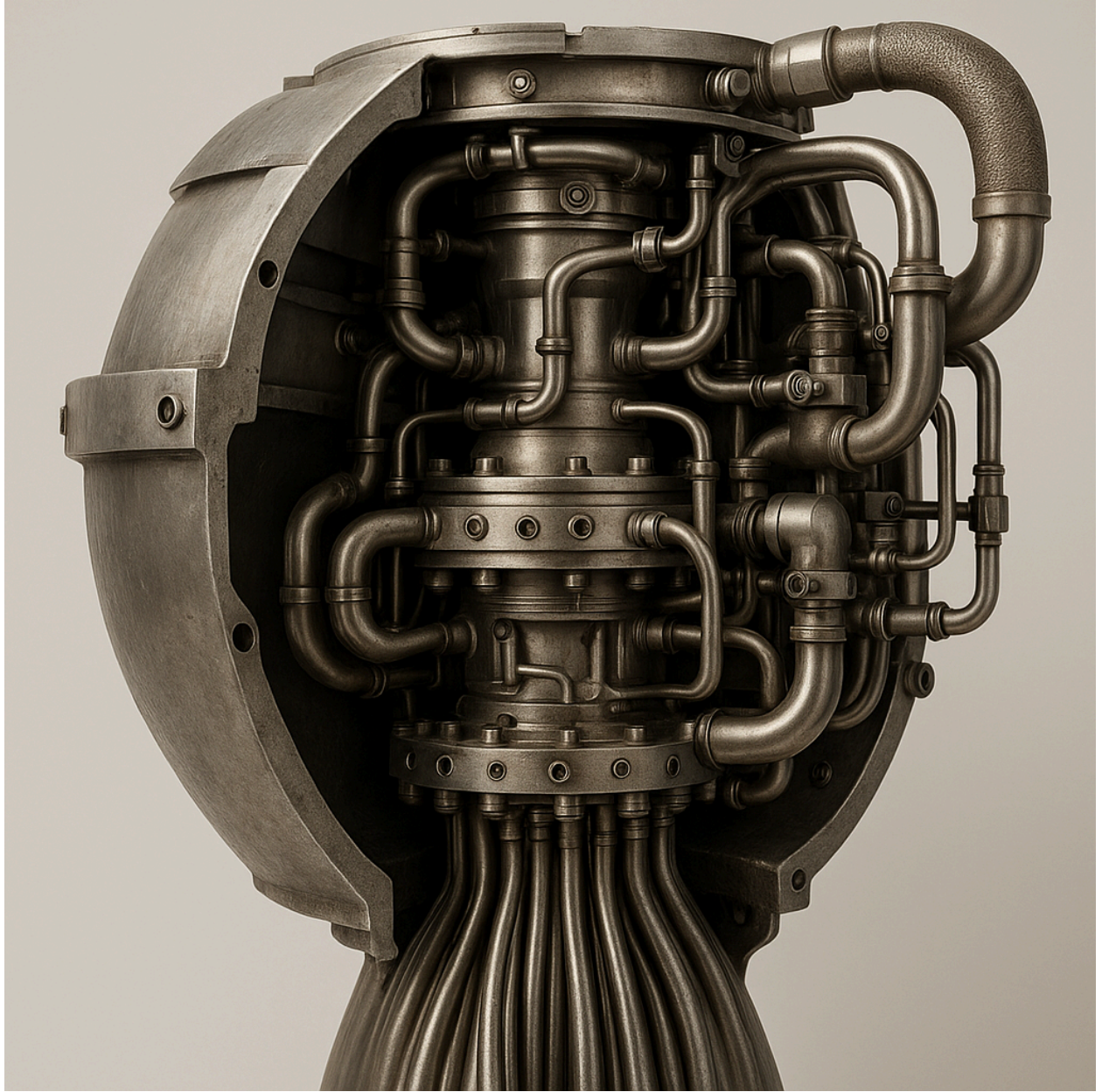
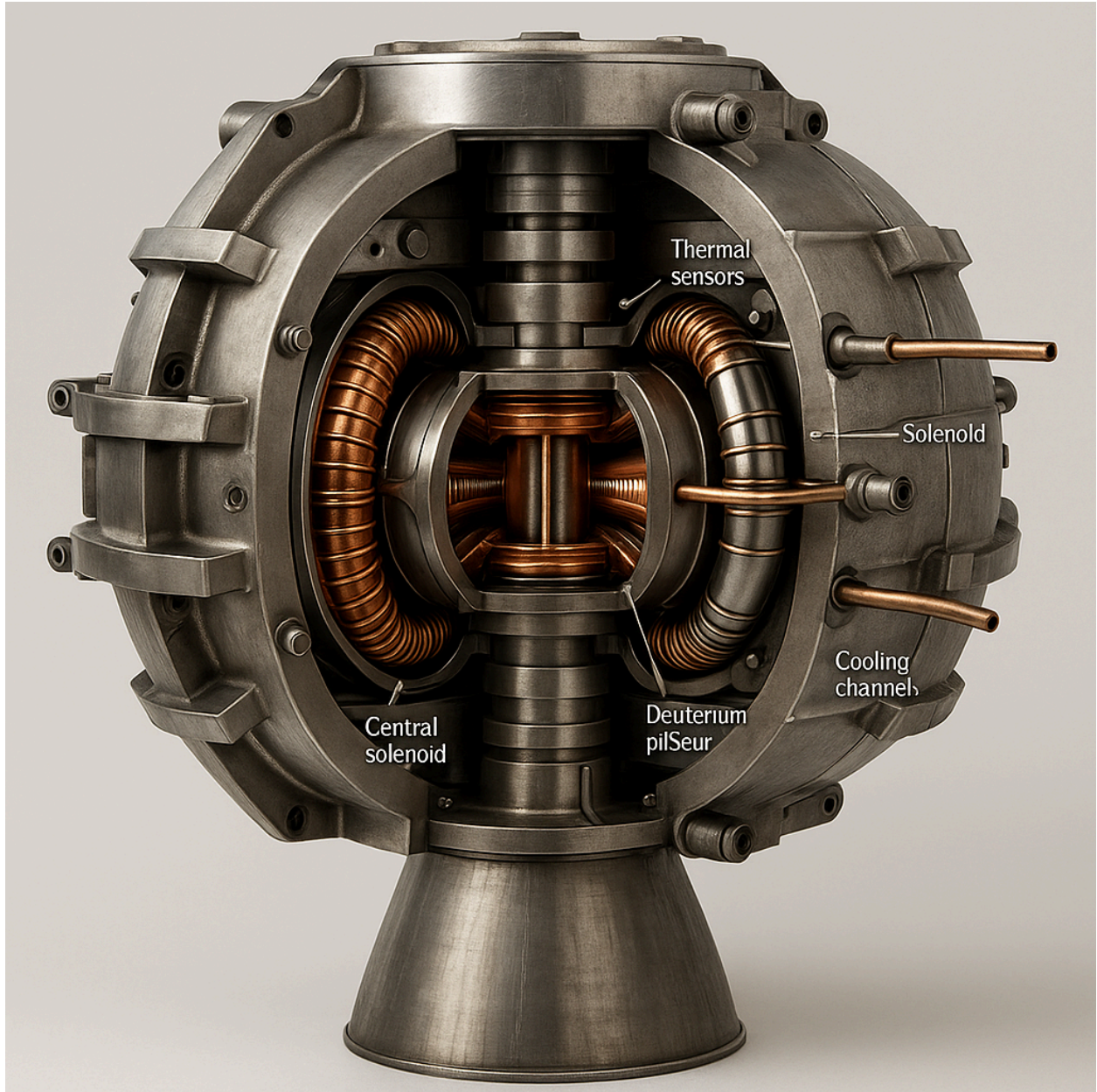
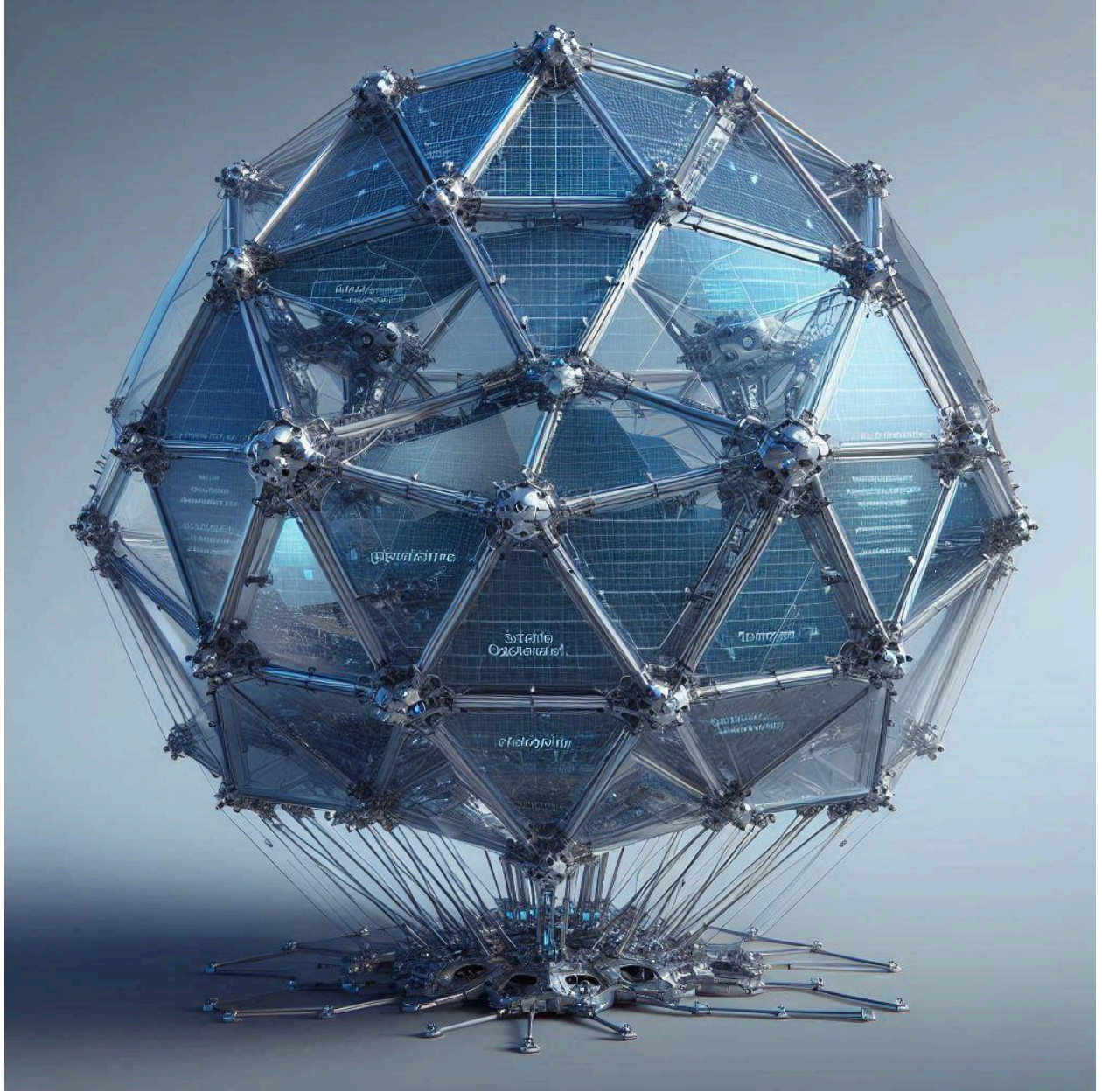


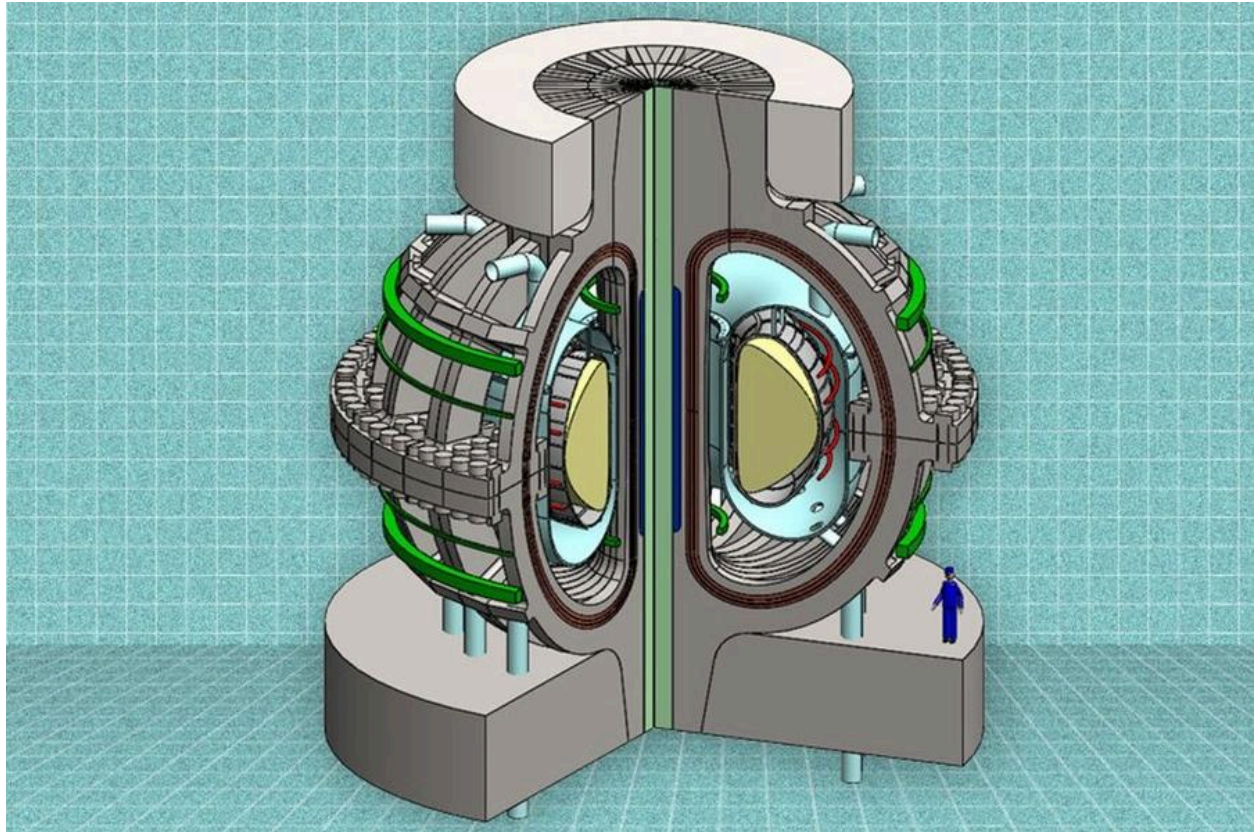


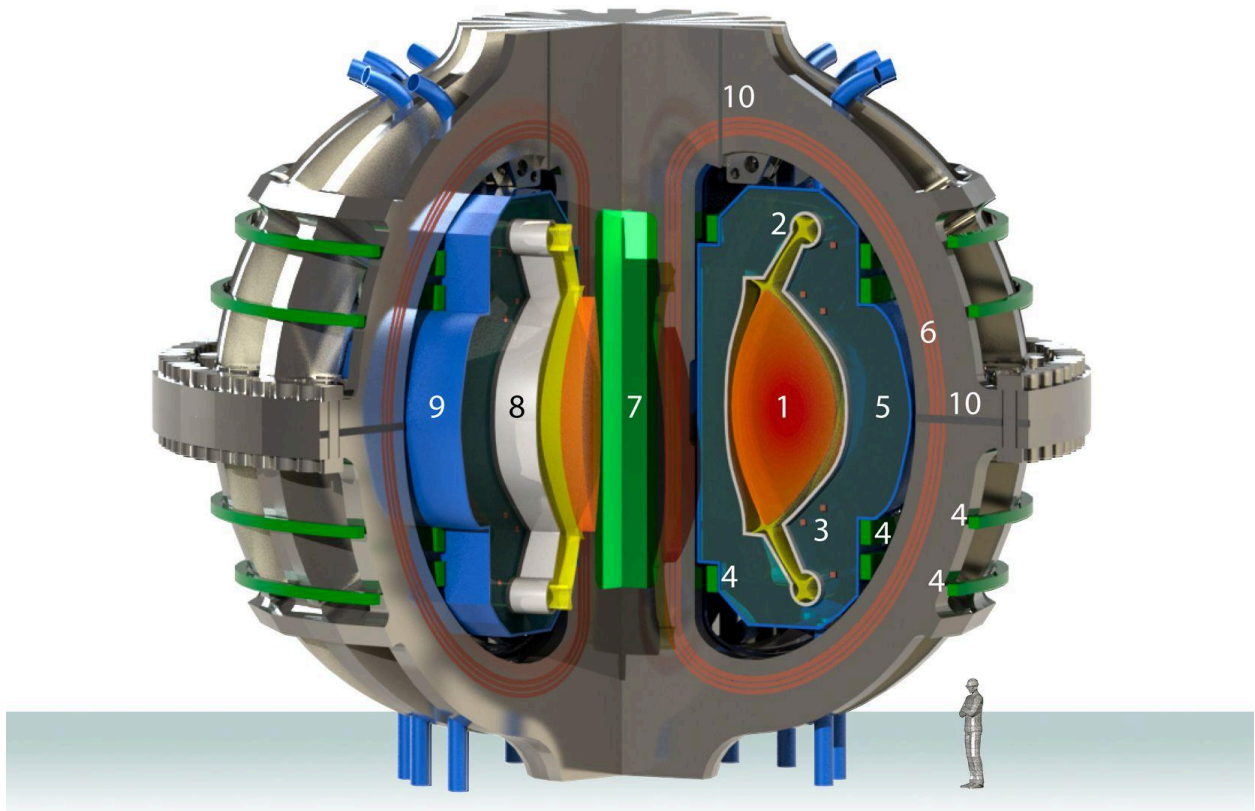


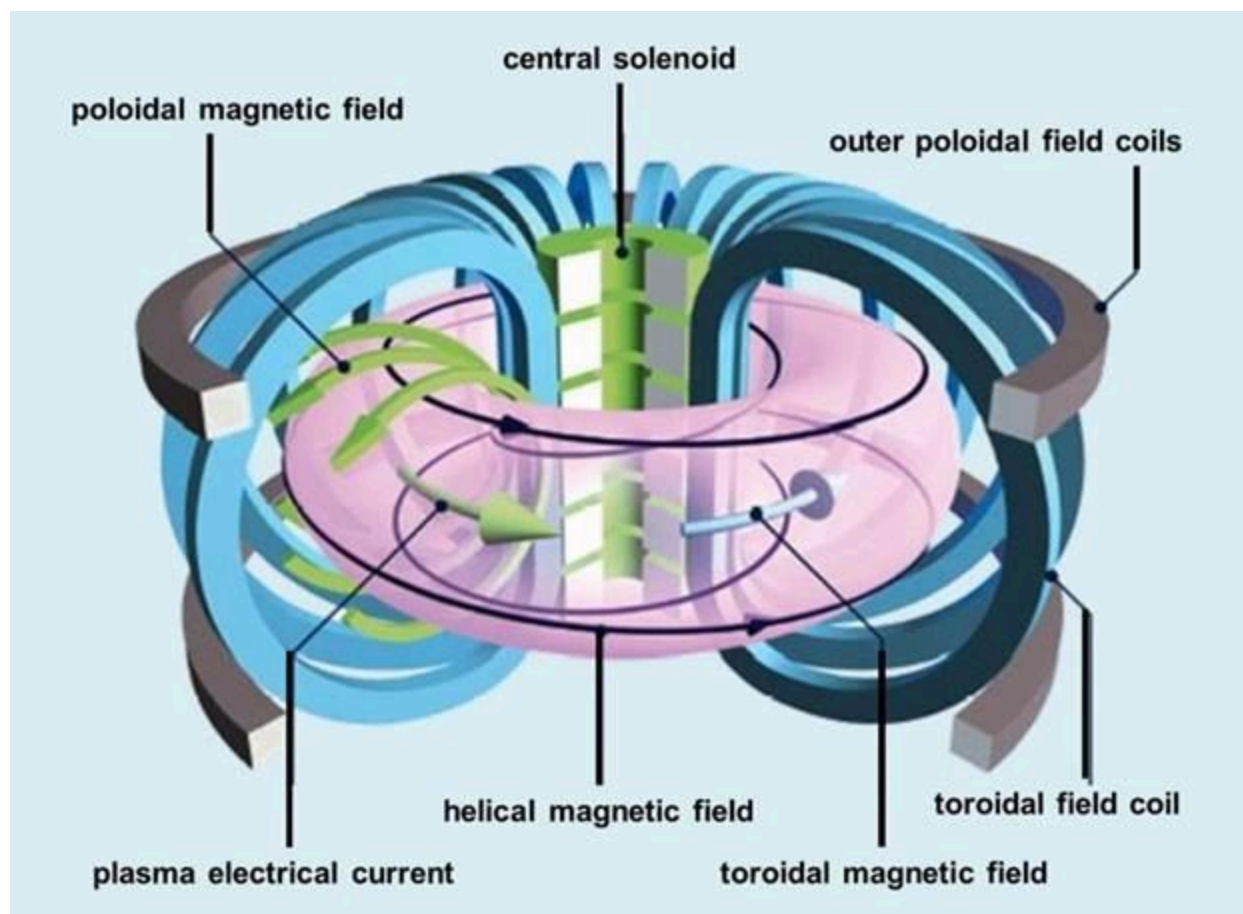


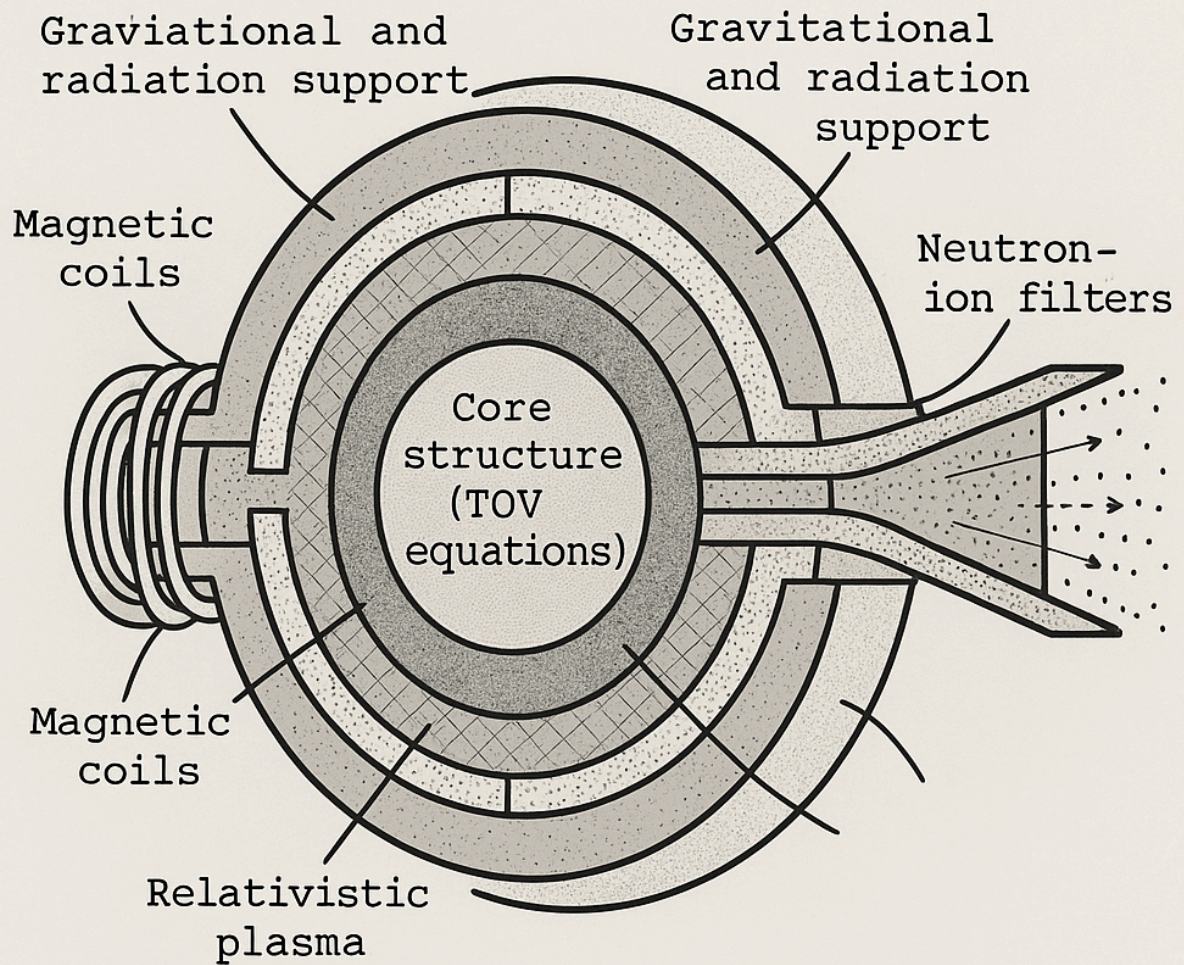




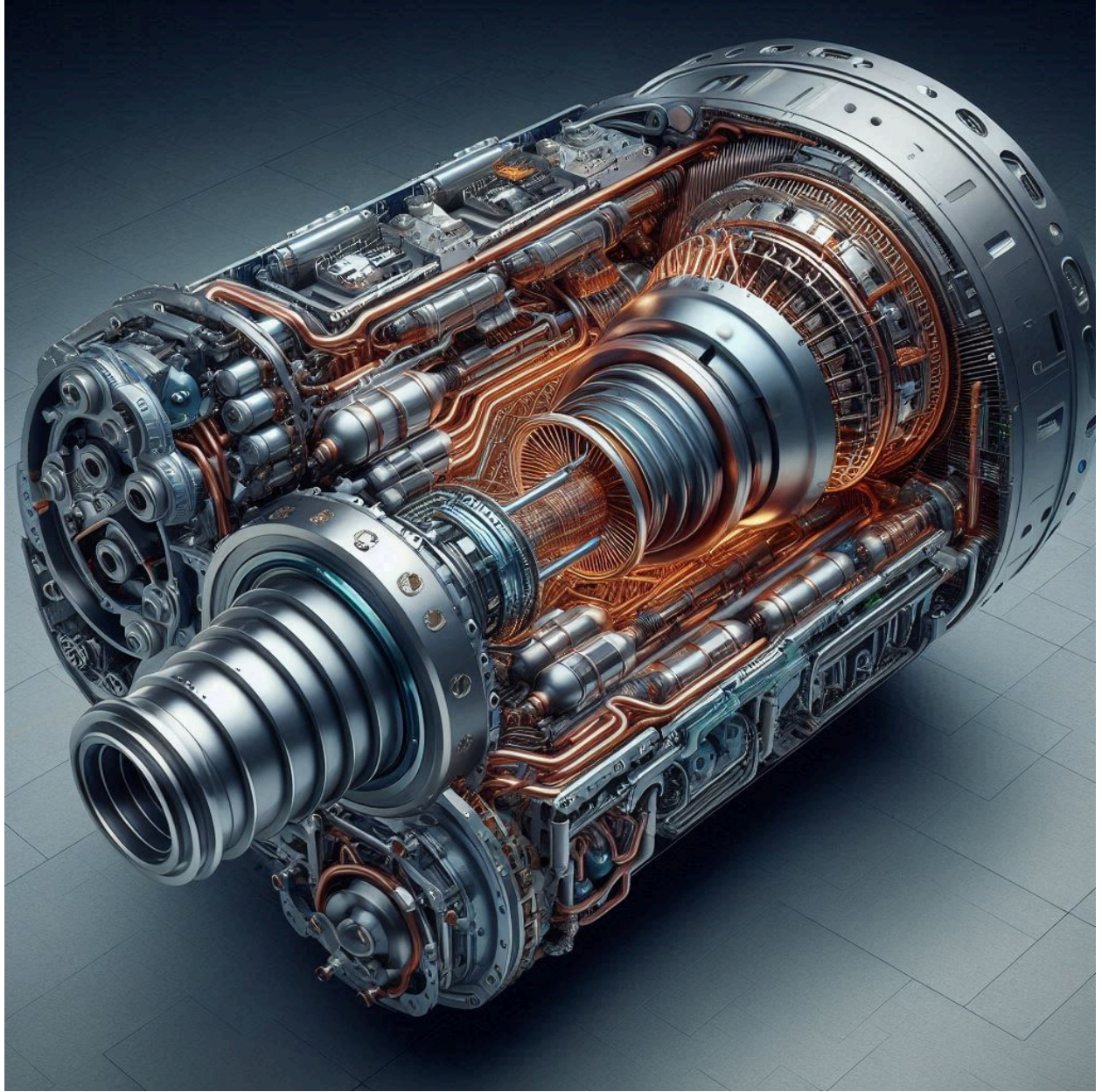


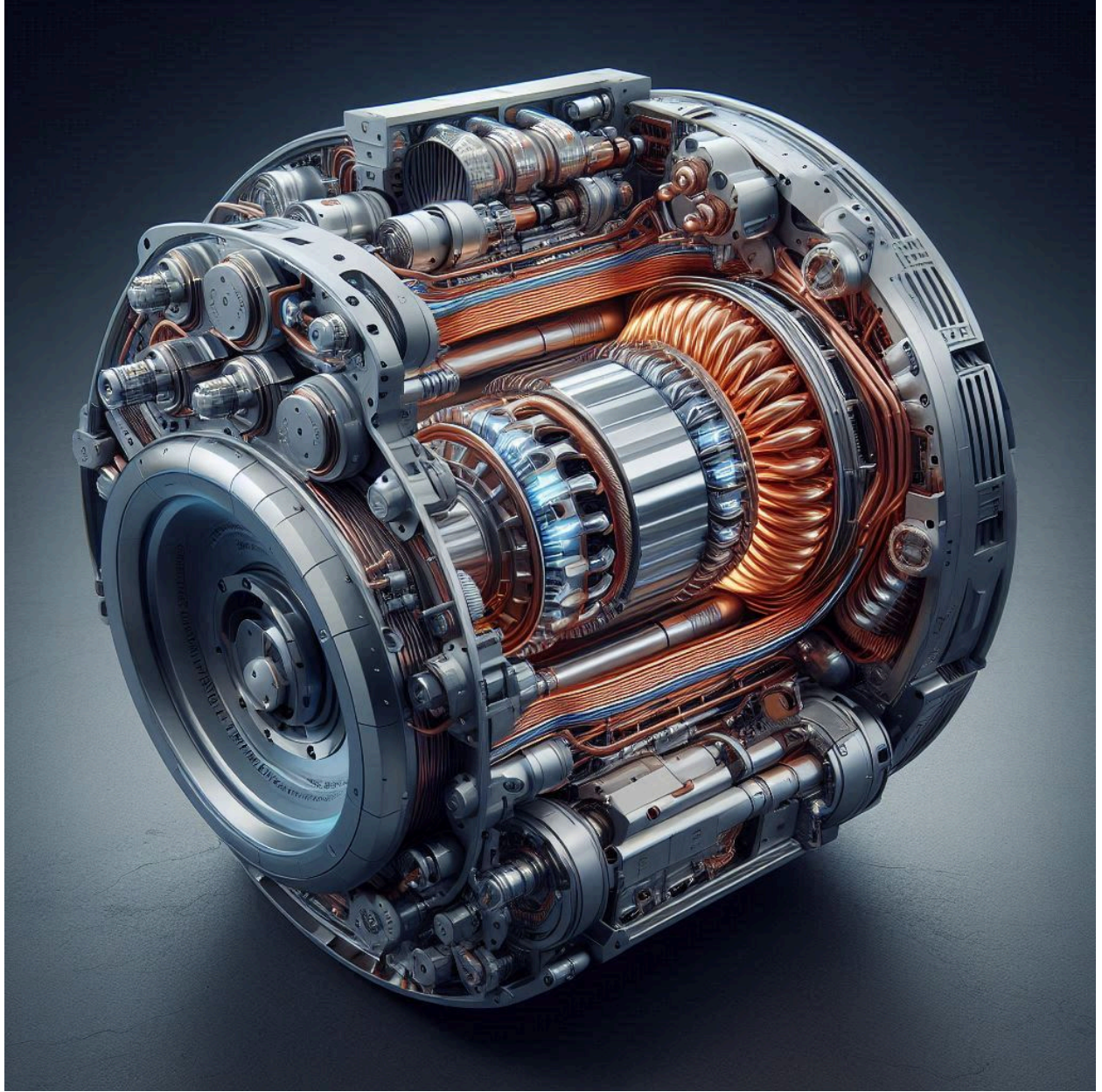


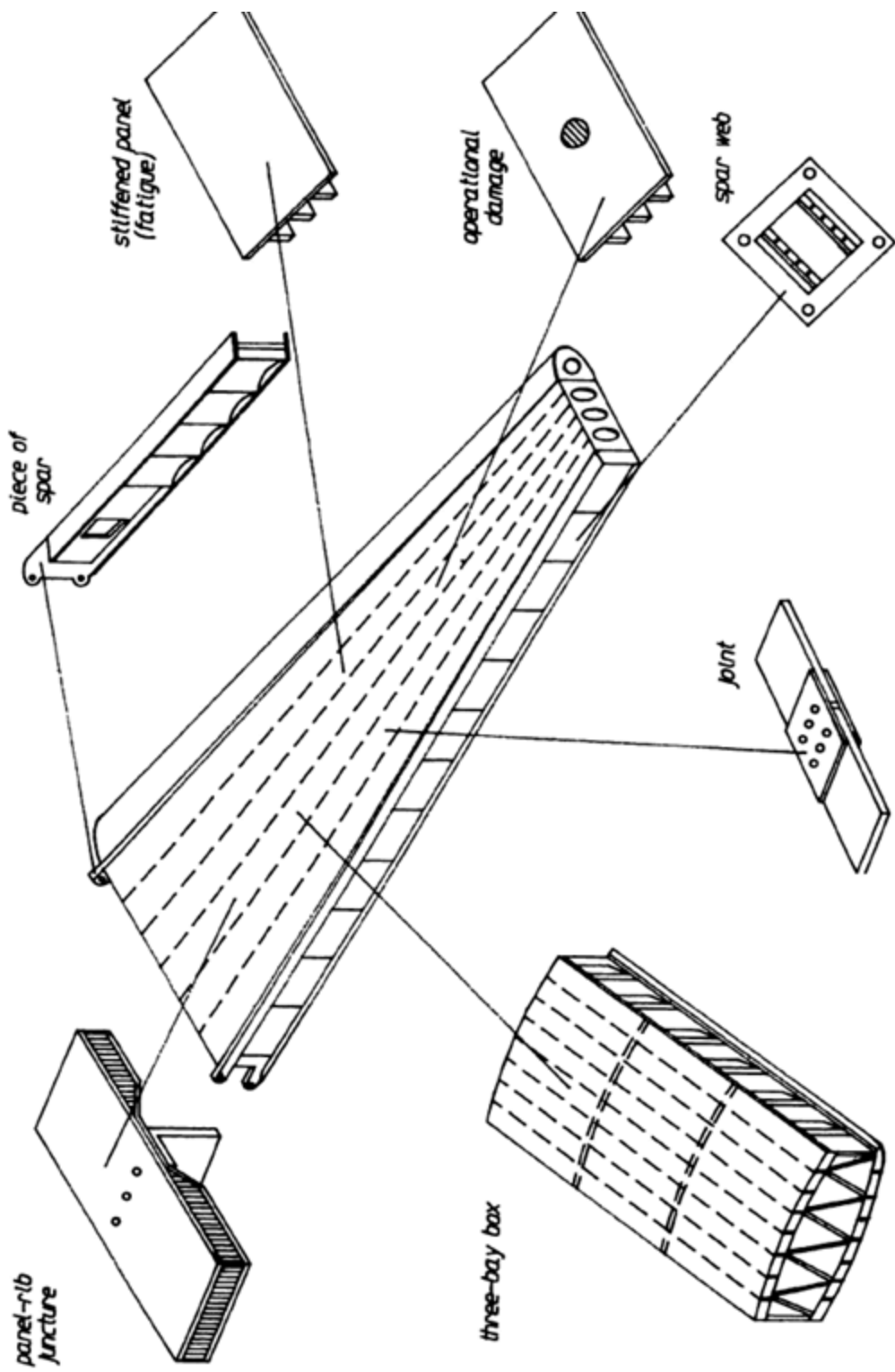


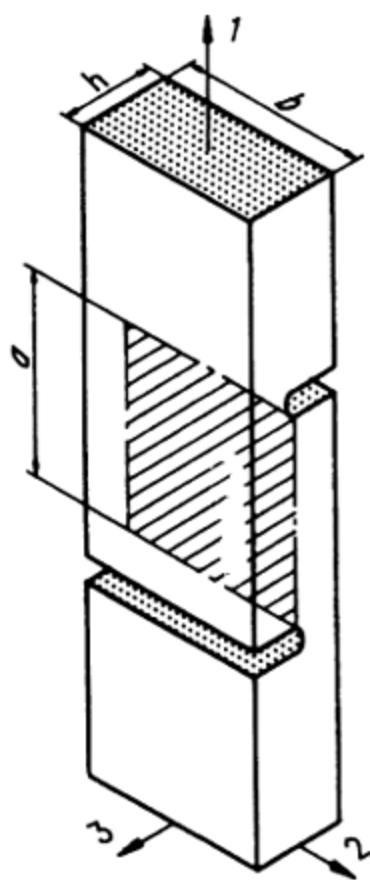


Relativistic plasma reactor with neutron-ion filtering, designed to withstand extreme gravity and radiation

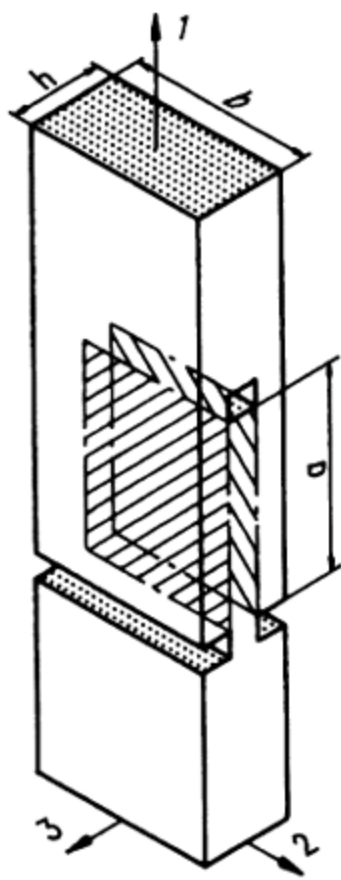






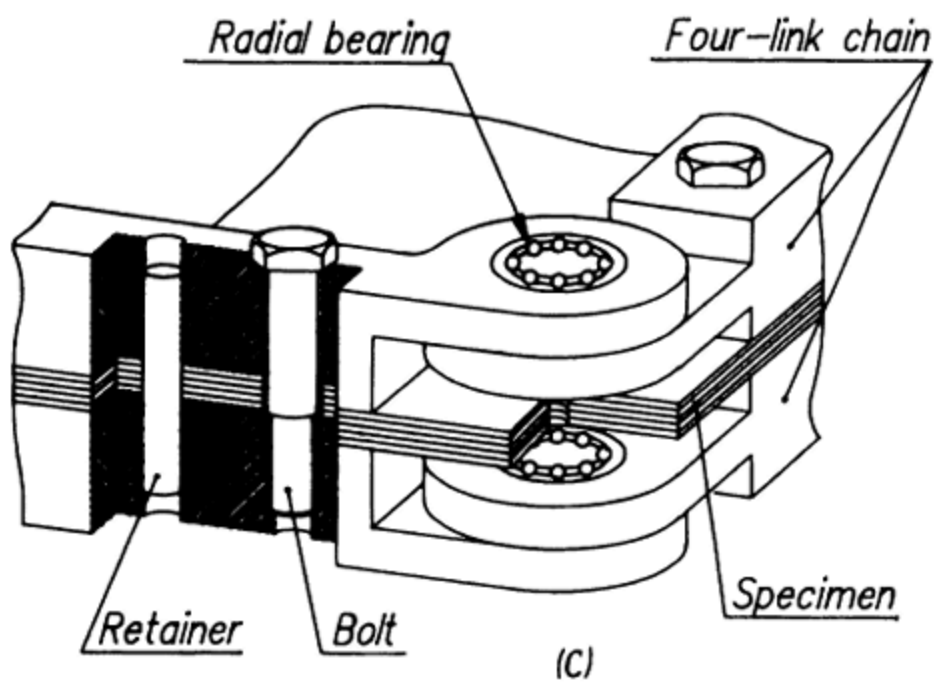


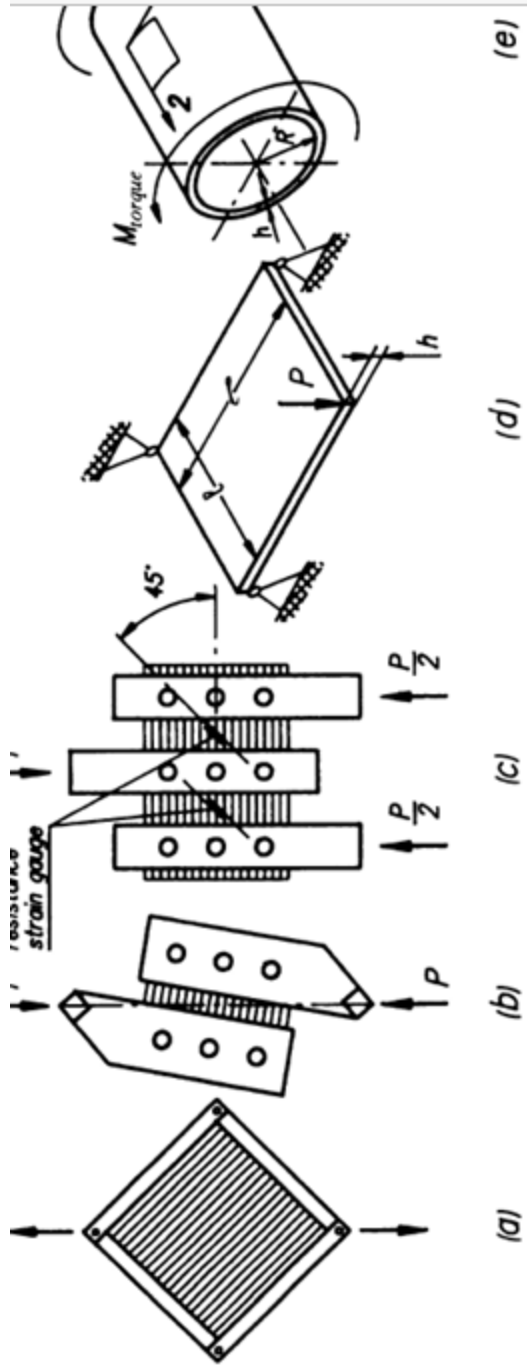
(a)

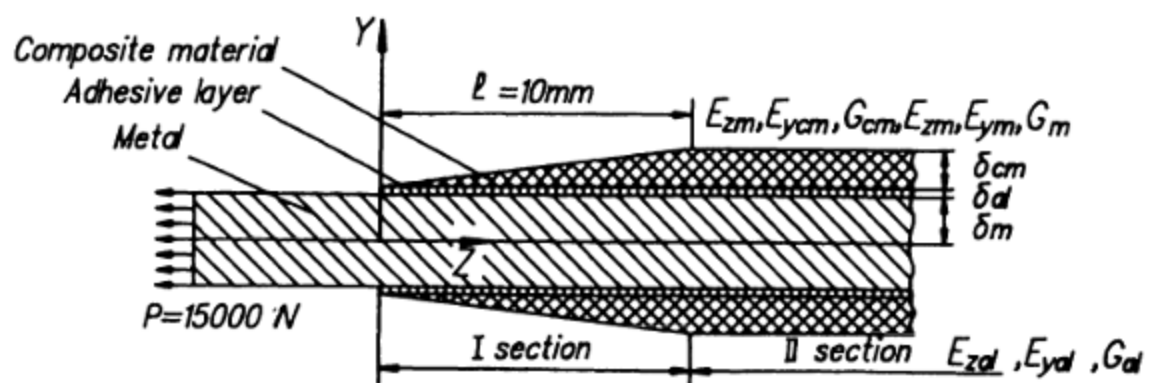


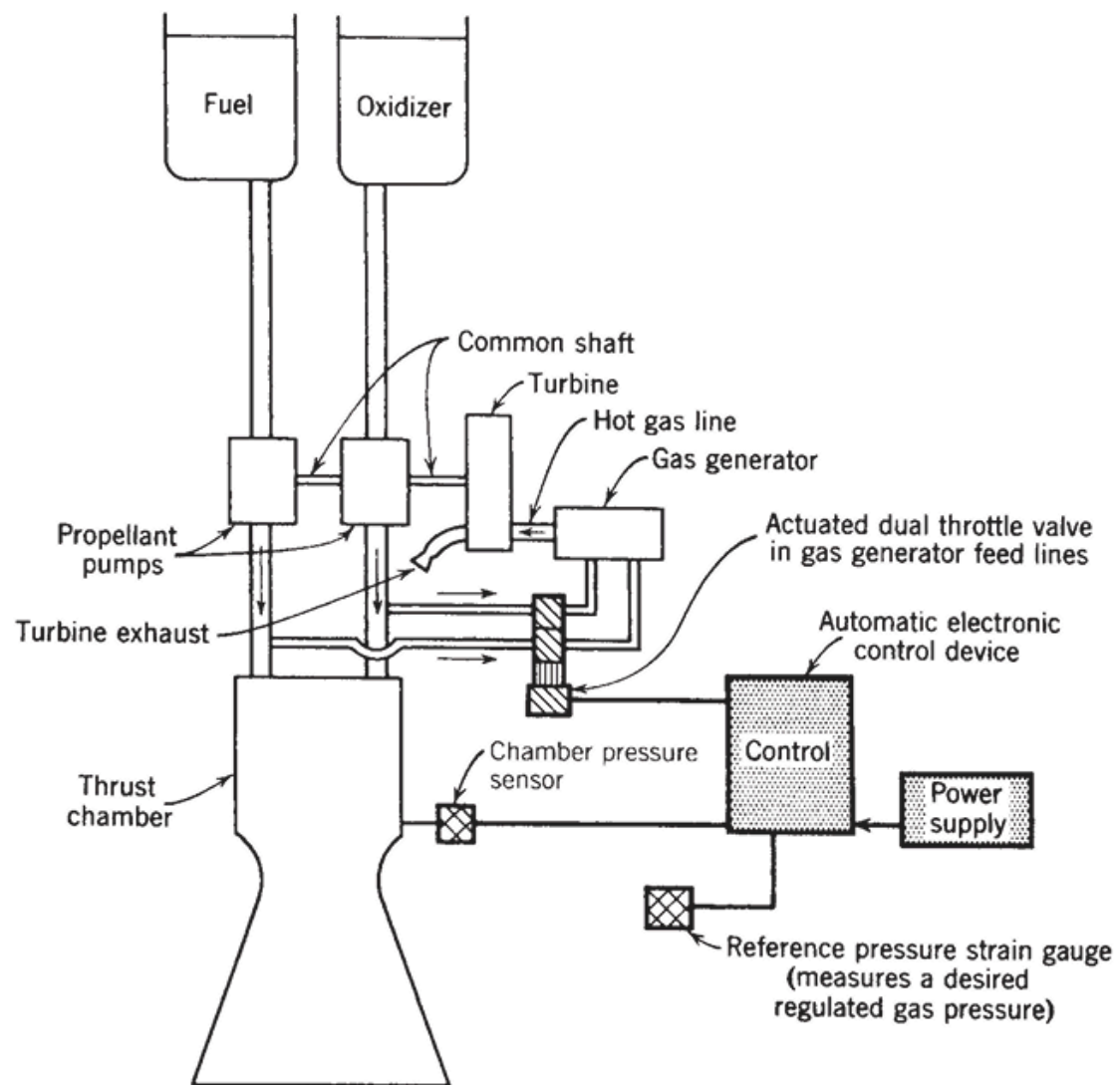
(b)

Detail I









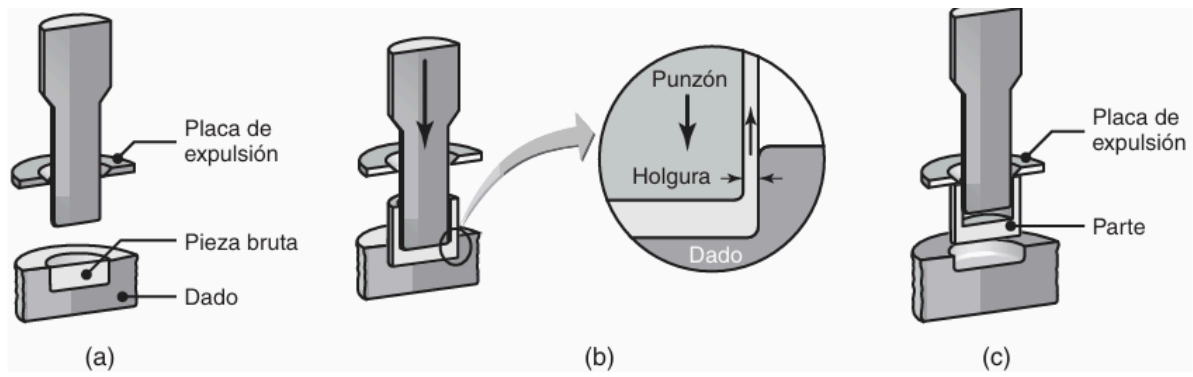
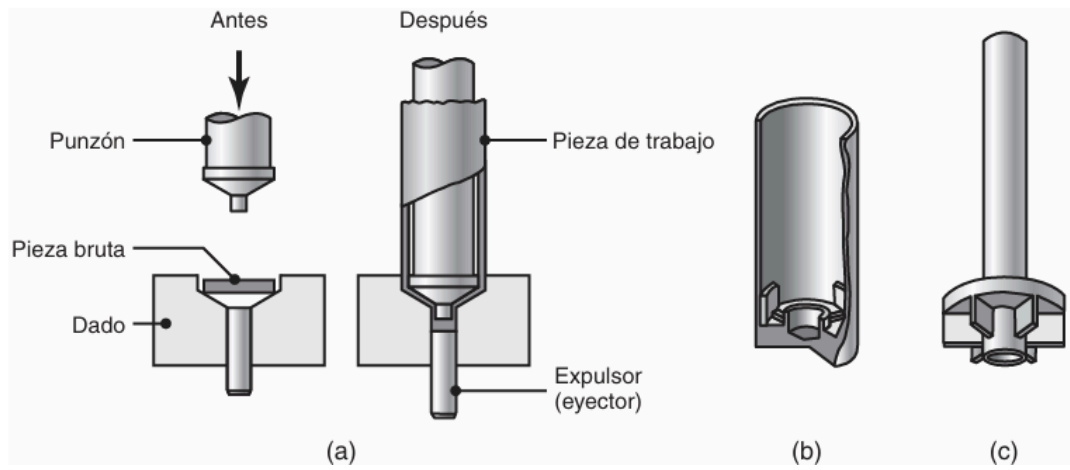
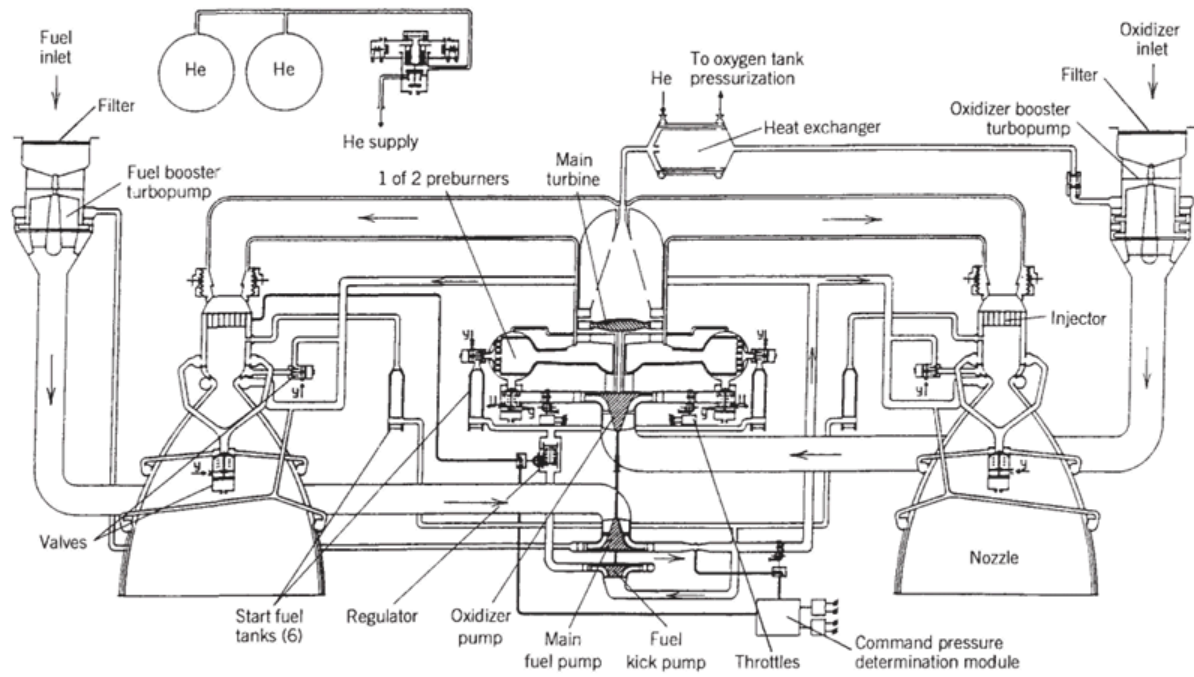
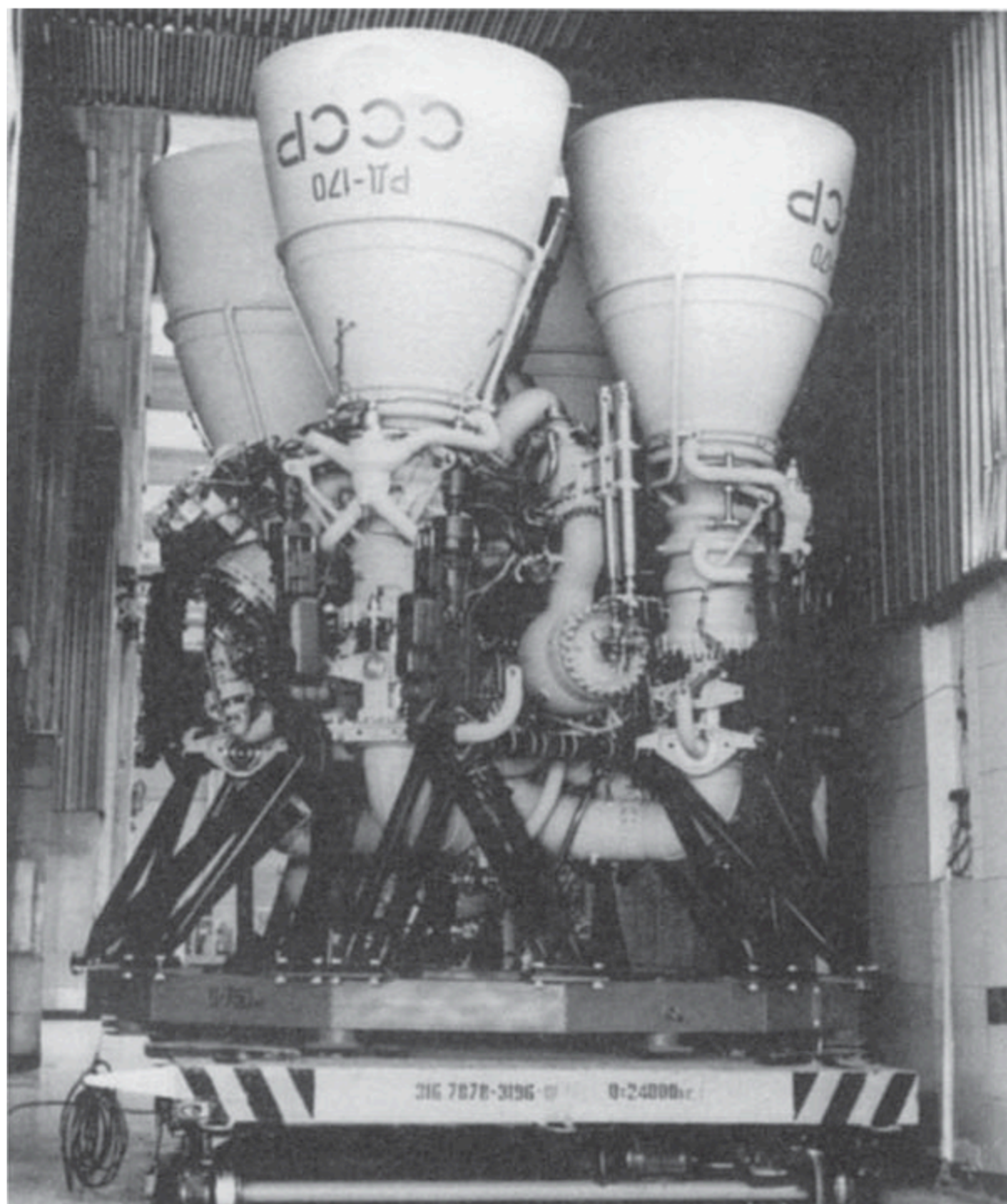
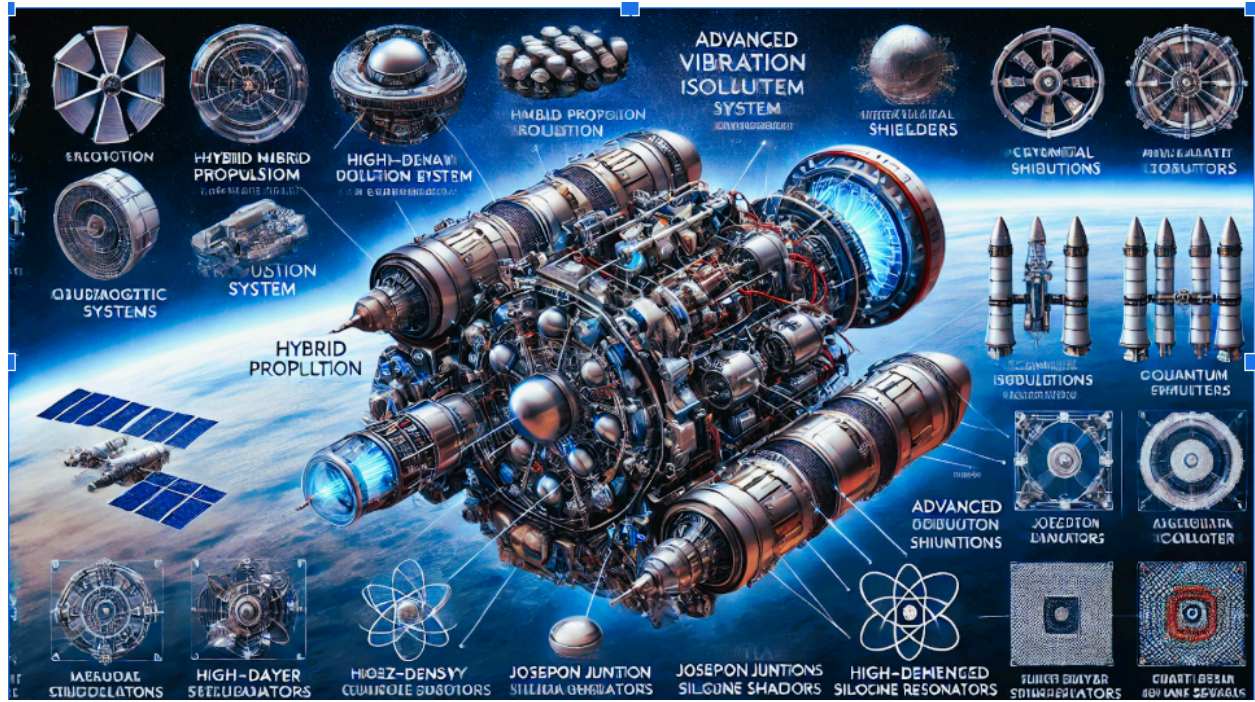


FIGURA 15.14 Esquema del proceso de extrusión por impacto. Las partes extruidas se expulsan mediante el uso de una placa de expulsión, ya que tienden a adherirse al punzón.

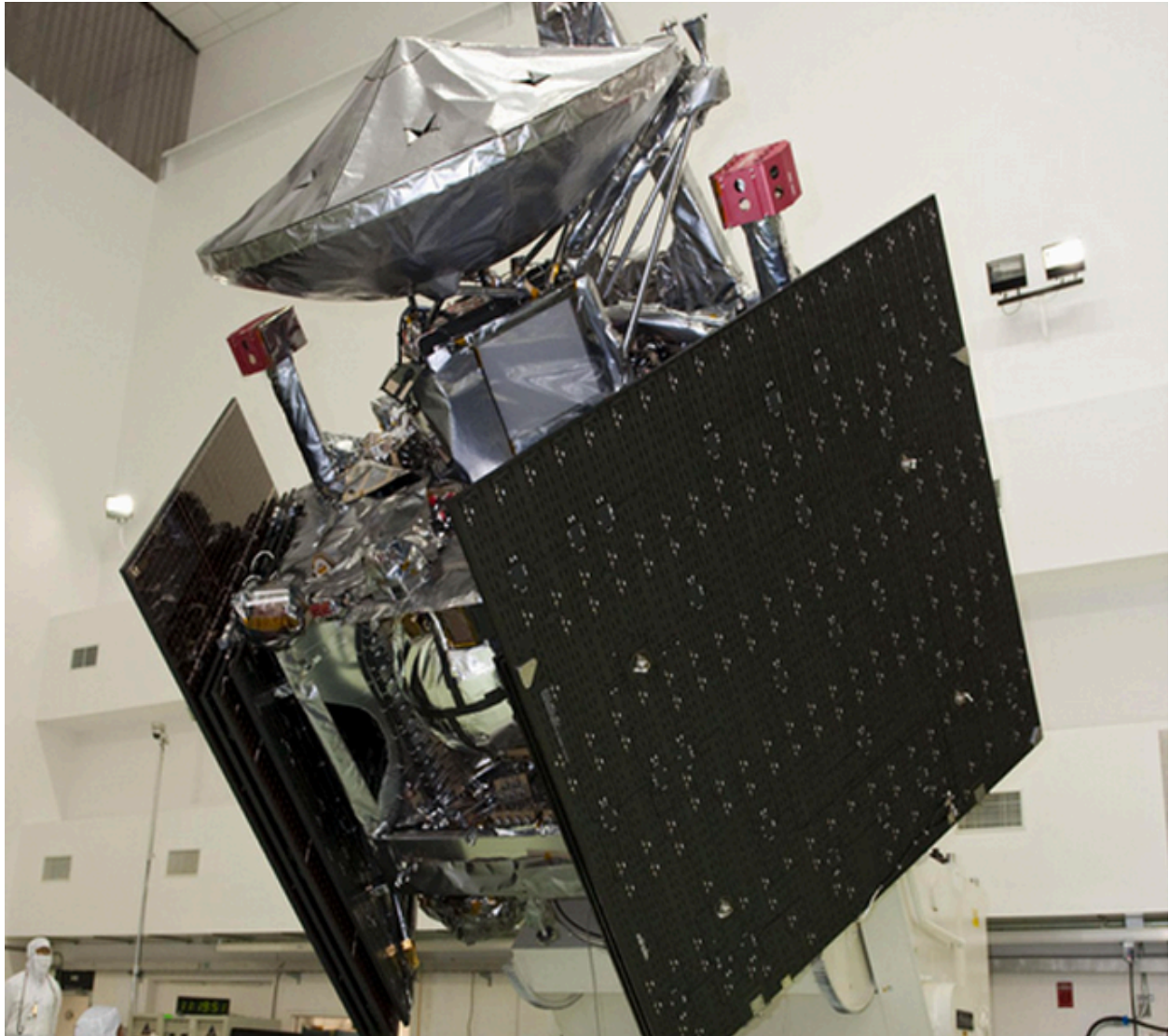












CODE

```
import bpy
import math
import mathutils

# Clear the scene
bpy.ops.object.select_all(action='SELECT')
bpy.ops.object.delete(use_global=False)

# Base scale
SCALE = 1.0

# === Nozzle style Merlin9 ===
bpy.ops.mesh.primitive_cone_add(
```



```

        radius1=0.9 * SCALE, radius2=0.25 * SCALE,
        depth=2.0 * SCALE, location=(0, 0, 0)
    )
    merlin_nozzle = bpy.context.object
    merlin_nozzle.name = "Merlin_Nozzle"

# === Tokamak Fusion Chamber ===
bpy.ops.mesh.primitive_torus_add(
    major_radius=0.75 * SCALE,
    minor_radius=0.15 * SCALE,
    location=(0, 0, 1.5 * SCALE),
    rotation=(0, 0, 0)
)
tokamak_chamber = bpy.context.object
tokamak_chamber.name = "Tokamak_Chamber"

# === Central Solenoid ===
bpy.ops.mesh.primitive_cylinder_add(
    radius=0.08 * SCALE, depth=1.4 * SCALE,
    location=(0, 0, 1.5 * SCALE)
)
solenoid = bpy.context.object
solenoid.name = "Central_Solenoid"

# === Toroidal coils ===
for angle in range(0, 360, 30):
    x = 0.75 * math.cos(math.radians(angle))
    y = 0.75 * math.sin(math.radians(angle))
    bpy.ops.mesh.primitive_torus_add(
        major_radius=0.05 * SCALE, minor_radius=0.015 * SCALE,
        location=(x, y, 1.5 * SCALE),
        rotation=(math.pi/2, 0, 0)
    )

# === Cooling ducts ===
for i in range(4):
    angle = math.radians(i * 90)
    x = 0.6 * math.cos(angle)
    y = 0.6 * math.sin(angle)
    bpy.ops.mesh.primitive_cylinder_add(
        radius=0.015 * SCALE, depth=0.5 * SCALE,
        location=(x, y, 1.8 * SCALE),
        rotation=(math.pi / 2, 0, angle)
    )

# === Deuterium Injector ===
bpy.ops.mesh.primitive_cylinder_add(
    radius=0.02 * SCALE, depth=0.4 * SCALE,
    location=(0.9 * SCALE, 0, 1.6 * SCALE),
    rotation=(0, math.pi / 2, 0)
)

```

```

# === External Heat Shield ===
bpy.ops.mesh.primitive_cylinder_add(
    radius=1.05 * SCALE, depth=0.4 * SCALE,
    location=(0, 0, 1.5 * SCALE)
)
shield_outer = bpy.context.object
bpy.ops.mesh.primitive_cylinder_add(
    radius=0.95 * SCALE, depth=0.4 * SCALE,
    location=(0, 0, 1.5 * SCALE)
)
bpy.ops.object.modifier_add(type='BOOLEAN')
shield_outer.modifiers['Boolean'].object = bpy.context.object
shield_outer.modifiers['Boolean'].operation = 'DIFFERENCE'
bpy.ops.object.modifier_apply(modifier="Boolean")

# === Control Node ===
bpy.ops.mesh.primitive_cube_add(
    size=0.1 * SCALE,
    location=(-0.4 * SCALE, 0, 1.8 * SCALE)
)

# === Basic materials (optional realism) ===
material_steel = bpy.data.materials.new(name="Steel")
material_steel.diffuse_color = (0.3, 0.3, 0.3, 1)

material_copper = bpy.data.materials.new(name="Copper")
material_copper.diffuse_color = (0.6, 0.3, 0.1, 1)

for obj in bpy.context.scene.objects:
    if "Nozzle" in obj.name:
        obj.data.materials.append(material_steel)
    elif "Solenoid" in obj.name or "Chamber" in obj.name:
        obj.data.materials.append(material_copper)

```

```

$fn = 100;

// Nozzle Merlin
difference() {
    cone(h=2.0, r1=0.25, r2=0.9);
    translate([0, 0, -0.1]) cylinder(h=2.1, r=0.2);
}

// Toroide Tokamak
translate([0, 0, 2.0])
    torus(R=0.75, r=0.15);

```

```

// Central solenoid
translate([0, 0, 2.0])
  cylinder(h=1.4, r=0.08);

// Toroidal coils
for (a = [0 : 30 : 330]) {
  rotate([0, 0, a])
  translate([0.75, 0, 2.0])
    rotate([90, 0, 0])
    torus(R=0.05, r=0.015);
}

// Injector
translate([0.9, 0, 2.2]) rotate([0, 90, 0]) cylinder(h=0.4, r=0.02);

// Ducts
for (a = [0 : 90 : 270]) {
  rotate([0, 0, a])
  translate([0.6, 0, 2.3]) cylinder(h=0.5, r=0.015);
}

// Heat shield
difference() {
  translate([0, 0, 1.9]) cylinder(h=0.4, r=1.05);
  translate([0, 0, 1.9]) cylinder(h=0.4, r=0.95);
}

// Control node
translate([-0.4, 0, 2.3]) cube([0.1, 0.1, 0.1], center=true);

module torus(R, r) {
  rotate_extrude(angle = 360)
    translate([R, 0, 0])
      circle(r);
}

```

Sim

```

# We will set up and create a basic simulation of a "Tokamak-Merlin"
type 3D model using Trimesh
# This script creates the primitives and groups them into a scene to
visualize the hybrid engine.
import trimesh
import numpy as np

# Create the Merlin-type nozzle (truncated cone)

```



```

nozzle_height = 2.0
nozzle_top_radius = 0.25
nozzle_bottom_radius = 0.9
nozzle_cone = trimesh.creation.cone(radius=nozzle_bottom_radius,
height=nozzle_height, sections=64)
nozzle_cone.apply_translation([0, 0, nozzle_height / 2])

# Toroidal chamber type Tokamak
tokamak_major_radius = 0.75
tokamak_minor_radius = 0.15
tokamak_torus = trimesh.creation.torus(radius=tokamak_major_radius,
tube_radius=tokamak_minor_radius)
tokamak_torus.apply_translation([0, 0, 2.0])

# Central solenoid (vertical cylinder)
solenoid = trimesh.creation.cylinder(radius=0.08, height=1.4,
sections=32)
solenoid.apply_translation([0, 0, 2.0])

# Toroidal coils (small tori around)
coils = []
for angle_deg in range(0, 360, 30):
    angle_rad = np.radians(angle_deg)
    x = tokamak_major_radius * np.cos(angle_rad)
    y = tokamak_major_radius * np.sin(rad_angle)
    coil = trimesh.creation.torus(radius=0.05, tube_radius=0.015)
    coil.apply_translation([x, y, 2.0])
    coil.apply_rotation(trimesh.transformations.rotation_matrix(np.pi
/ 2, [1, 0, 0]))
    bobinas.append(coil)

# Deuterium injector (side cylinder)
injector = trimesh.creation.cylinder(radius=0.02, height=0.4,
sections=32)
injector.apply_translation([0.9, 0, 2.2])
injector.apply_transform(trimesh.transformations.rotation_matrix(np.p
i / 2, [0, 1, 0]))

# Cooling ducts
ducts = []
for i in range(4):
    angle_rad = np.radians(i * 90)
    x = 0.6 * np.cos(angle_rad)
    y = 0.6 * np.sin(angle_rad)
    tube = trimesh.creation.cylinder(radius=0.015, height=0.5,
sections=16)
    tube.apply_translation([x, y, 2.3])
    tube.apply_rotation(trimesh.transformations.rotation_matrix(np.pi
/ 2, [1, 0, 0]))
    ducts.append(tube)

```

```

# Create the scene
scene = trimesh.Scene()
scene.add_geometry(nozzle_cone, node_name="Nozzle")
scene.add_geometry(tokamak_torus, node_name="Tokamak")
scene.add_geometry(solenoid, node_name="Solenoid")
scene.add_geometry(injector, node_name="Injector")
for i, coil in enumerate(bobinas):
    scene.add_geometry(coil, node_name=f"Coil_{i}")
for i, tube in enumerate(conductos):
    scene.add_geometry(tube, node_name=f"Tube_{i}")

scene.show()

```

Brave

```

import trimesh
import numpy as np

def create_torus(R=0.75, r=0.15, segments=64):
    theta = np.linspace(0, 2 * np.pi, segments)
    phi = np.linspace(0, 2 * np.pi, segments)
    theta, phi = np.meshgrid(theta, phi)
    x = (R + r * np.cos(theta)) * np.cos(phi)
    y = (R + r * np.cos(theta)) * np.sin(phi)
    z = r * np.sin(theta)
    vertices = np.stack([x, y, z], axis=-1).reshape(-1, 3)
    faces = []
    for i in range(segments):
        for j in range(segments):
            i0 = i * segments + j
            i1 = ((i + 1) % segments) * segments + j
            i2 = ((i + 1) % segments) * segments + (j + 1) % segments
            i3 = i * segments + (j + 1) % segments
            faces.append([i0, i1, i2])
            faces.append([i0, i2, i3])
    return trimesh.Trimesh(vertices=vertices, faces=faces)

scene = trimesh.Scene()

# The nozzle Merlin
nozzle = trimesh.creation.cone(radius=0.9, height=2.0, sections=64)
nozzle.apply_translation([0, 0, 1.0])
scene.add_geometry(nozzle, node_name="Nozzle")

# Toroide Tokamak
tokamak_torus = create_torus()
tokamak_torus.apply_translation([0, 0, 3.0])

```

```

scene.add_geometry(tokamak_torus, node_name="Tokamak")

# Solenoid
solenoid = trimesh.creation.cylinder(radius=0.08, height=1.4)
solenoid.apply_translation([0, 0, 3.0])
scene.add_geometry(solenoid, node_name="Solenoid")

# Toroidal coils
for angle_deg in range(0, 360, 30):
    angle_rad = np.radians(angle_deg)
    x = 0.75 * np.cos(angle_rad)
    y = 0.75 * np.sin(angle_rad)
    coil = create_torus(R=0.05, r=0.015)
    coil.apply_translation([x, y, 3.0])

coil.apply_transform(trimesh.transformations.rotation_matrix(np.pi /
2, [1, 0, 0]))
    scene.add_geometry(coil, node_name=f"Coil_{angle_deg}")

# Deuterium injector
injector = trimesh.creation.cylinder(radius=0.02, height=0.4)
injector.apply_translation([0.9, 0, 3.3])
injector.apply_transform(trimesh.transformations.rotation_matrix(np.p
i / 2, [0, 1, 0]))
scene.add_geometry(injector, node_name="Injector")

# Cooling ducts
for i in range(4):
    angle = np.radians(i * 90)
    x = 0.6 * np.cos(angle)
    y = 0.6 * np.sin(angle)
    tube = trimesh.creation.cylinder(radius=0.015, height=0.5)
    tube.apply_translation([x, y, 3.5])

tube.apply_transform(trimesh.transformations.rotation_matrix(np.pi /
2, [1, 0, 0]))
    scene.add_geometry(tube, node_name=f"Tube_{i}")

scene.show()

```

Geometry/

```

import bpy
import math
from math import pi

# Clear scene

```



```

bpy.ops.object.select_all(action='SELECT')
bpy.ops.object.delete(use_global=False)

# Auxiliary function: create toroid
def add_torus(name, major, minor, location, rotation=(0, 0, 0)):
    bpy.ops.mesh.primitive_torus_add(
        major_radius=major,
        minor_radius=minor,
        location=location,
        rotation=rotation,
        abso_major_rad=1.25,
        abso_minor_rad=0.75)
    obj = bpy.context.object
    obj.name = name

# Merlin nozzle (inverted cone)
bpy.ops.mesh.primitive_cone_add(
    vertices=128,
    radius1=0.85,
    radius2=0.25,
    depth=2.0,
    location=(0, 0, 1.0))
bpy.context.object.name = "Merlin_Nozzle"

# Toroidal chamber toroidal tortoise
add_torus("Tokamak_Core", major=0.75, minor=0.15, location=(0, 0, 3.0))

# Central solenoid
bpy.ops.mesh.primitive_cylinder_add(
    radius=0.08,
    depth=1.5,
    location=(0, 0, 3.0))
bpy.context.object.name = "Solenoid_Center"

# Toroidal coils (x12)
for angle in range(0, 360, 30):
    rad = math.radians(angle)
    x = 0.75 * math.cos(rad)
    y = 0.75 * math.sin(rad)
    add_torus(
        name=f"Toroidal_Coil_{angle}",
        major=0.05,
        minor=0.015,
        location=(x, y, 3.0),
        rotation=(pi / 2, 0, 0))

# Deuterium injector
bpy.ops.mesh.primitive_cylinder_add(
    radius=0.02,
    depth=0.4,
    location=(0.9, 0, 3.3),

```

```

        rotation=(0, pi/2, 0))
bpy.context.object.name = "Deuterium_Injector"

# Cooling ducts (4 symmetrical)
for i in range(4):
    angle = math.radians(i * 90)
    x = 0.6 * math.cos(angle)
    y = 0.6 * math.sin(angle)
    bpy.ops.mesh.primitive_cylinder_add(
        radius=0.015,
        depth=0.5,
        location=(x, y, 3.5),
        rotation=(pi/2, 0, 0))
    bpy.context.object.name = f"Coolant_Tube_{i}"

# External heat shield
bpy.ops.mesh.primitive_cylinder_add(
    radius=0.9,
    depth=0.5,
    location=(0, 0, 3.0))
outer = bpy.context.object
outer.name = "Thermal_Shield"

bpy.ops.mesh.primitive_cylinder_add(
    radius=0.75,
    depth=0.5,
    location=(0, 0, 3.0))
inner = bpy.context.object
inner.name = "Thermal_Shield_Inner"

# Perform boolean difference (internal gap)
mod_bool = outer.modifiers.new(name="cut", type='BOOLEAN')
mod_bool.operation = 'DIFFERENCE'
mod_bool.object = inner
bpy.context.view_layer.objects.active = outer
bpy.ops.object.modifier_apply(modifier="cut")
bpy.data.objects.remove(inner)

# Electronic control node
bpy.ops.mesh.primitive_cube_add(
    size=0.1,
    location=(-0.6, 0, 3.6))
bpy.context.object.name = "Control_Node"

```

1. Inverted Merlin nozzle titanium
2. Tokamak Toroidal Camera
3. Solenoid vertical
4. Rotated toroidal coils
5. Deuterium injector
6. Symmetrical cooling ducts
7. Metal heat shield with gap
8. Electronic control box

hydrogen/deuterium chamber

Fuel storage and regulation

- Deuterium is not injected directly from the outside. A pressurized storage system containing deuterium gas or solid pellets is required.
- This system must allow controlled and precise injection to maintain plasma density within operating margins ($\sim 10^{19}$ particles/m³ in many reactors).

Warm-up and conditioning

- In some designs, the gas goes through thermal conditioning or partial ionization stages before entering the tokamak.
- This is done to improve the efficiency of ionization within the toroid.

Synchronization with magnetic fields

- The injection system (pellet or gas puffing) must be coordinated with the magnetic system to prevent the fuel from destabilizing the plasma.

Hydrogen chamber design elements

the elements you could model in OpenSCAD or Blender:

Pressurized tank (fuel storage)

- External metal cylinder (stainless steel or titanium) with valves and regulators.
- It can be thermally insulated if it stores liquid or solid deuterium.

Injection valves

- High-precision solenoid valves mounted in line with the injectors.
- They are synchronized with the control system and the state of the plasma.

Distribution ducts

- Metal or ceramic tubes that carry the gas from the tank to the injector.
- They can include pressure, temperature and flow sensors.

Pellet injector or launcher

- Alternatively, a pellet injection system can be designed, where small solid pellets of deuterium are fired into the core.
- Ideal for re-injecting fuel without disturbing plasma stability.

Inverted Titanium Merlin Nozzle (Combustion Nozzle / Exhaust Bell)

The Merlin nozzle is typically a bell-shaped expansion chamber designed to convert thermal and pressure energy from combustion into directed thrust. In this hybrid design:

- Material: Titanium alloy is used for high-temperature resistance and weight efficiency.
- Function: Instead of chemical combustion, it exhausts heated plasma confined in the tokamak, creating thrust through magnetic-expelled plasma streams.

- Design Parameters: Expansion ratio, throat diameter, and diverging cone angle must be tuned to the specific energy profile of plasma exhaust.
-

Toroidal Tokamak Plasma Chamber

The tokamak is a torus-shaped chamber used to confine plasma via powerful magnetic fields:

- Vacuum Vessel: The core chamber where low-pressure deuterium gas is ionized into plasma.
- Magnetic Field: Created by external toroidal and poloidal coils to confine the plasma.
- Fusion Goal: Deuterium nuclei collide and fuse under thermal pressure, releasing high-energy neutrons.

Theory: The tokamak relies on the magnetic pinch effect and toroidal confinement to contain plasma, governed by the Grad–Shafranov equation and MHD stability criteria.

Vertical Solenoid (Central Solenoid)

The central solenoid is a key component in plasma ignition and confinement:

- Role: Induces a toroidal current within the plasma, enabling magnetic confinement (acts like a transformer).
 - Design: Composed of stacked superconducting coils for high magnetic field strength (~5–10 Tesla).
 - Operation: Pulsed induction aligns plasma particles and initiates the fusion process.
-

Rotated Toroidal Field Coils

These coils generate the toroidal component of the magnetic field:

- Positioning: Equally spaced around the torus for symmetric field distribution.
- Shape: Often D-shaped or circular to follow the vessel profile.
- Material: Typically copper or niobium-tin (Nb₃Sn) for superconductivity.

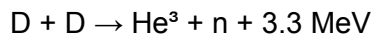
They stabilize the plasma, prevent drift, and sustain long confinement times for sustained fusion.

Deuterium Injector

Deuterium is the fusion fuel used in this system:

- Injection: Injected as gas or pellets into the chamber at timed intervals.
- Ionization: Exposed to radiofrequency (RF) fields or neutral beam injectors for ionization.
- Mass Flow Control: Regulated to maintain optimal density and fuel rate.

Fusion reactions:



Symmetrical Cooling Ducts (Thermal Channels)

Due to extreme heat from plasma and magnetic coils, thermal management is essential:

- Structure: Liquid-cooled copper or Inconel piping embedded around the vessel and nozzle walls.
 - Coolants: Can use liquid hydrogen, helium or water depending on pressure.
 - Purpose: Prevent thermal fatigue, structural degradation, and enable sustained operation.
-

Thermal Shield with Hollow Interior

The thermal shield protects structural elements and electronics from radiation and heat:

- Outer Shell: Made of high-thermal-capacity metals like tungsten, carbon composites or ceramic coating.
 - Inner Hollow: Allows for insulation, reflective layers, and mechanical expansion.
 - Design: Often uses a sandwich structure with multiple conductive/insulating layers.
-

Electronic Control Box

This module houses avionics and feedback systems:

- Includes: Real-time magnetic field regulators, thermal sensors, current monitors, PID controllers.
 - Architecture: Often FPGA-based or embedded RTOS with radiation-hardened components.
 - Interfaces: Communicates with flight control systems and external telemetry.
-

Supporting Documentation and Technical Resources

Fusion & Tokamak Design

- ITER Basics (<https://www.iter.org/mach>)
- MIT PSFC Tokamak Overview (<https://www.psfc.mit.edu/>)
- "Introduction to Plasma Physics and Controlled Fusion" – Francis F. Chen (PDFs available online)

Could fusion, the process that fuels the sun and stars, be a source of plentiful, clean, carbon-free energy on earth? Learn how fusion occurs, and what researchers around the world are doing to understand the process, and to harness its potential. Narrated by Plasma Science and Fusion Center.

Most magnetic confinement fusion reactors (tokamaks, stellarators) make use of high power radio frequency (RF) actuators for access to and control of the burning plasma. SPARC, for example, will use 20+ MW of RF power in the ion cyclotron range of frequencies (ICRF) at 120 MHz to heat the plasma up to temperatures where fusion reactions become the dominant heating source. The RF antennas must be located close to the edge plasma with a direct line-of-sight for efficient coupling of the waves, therefore effectively functioning as plasma facing components (PFCs). The heat loads on the plasma facing surfaces of the antenna are typically cooled with forced liquid or gas flow through cooling channels beneath the surface. Forced flow cooling introduces additional points of failure in an already complicated system, and decreases efficiency through pumping losses. Existing long-pulse fusion reactors typically use high pressure water as a coolant, however water cooling inside the vacuum vessel introduces significant detritiation complexity and safety risks in a DT reactor environment. Tritium can be more easily separated from gaseous helium coolant, however gas cooling is much less effective as compared to liquid cooling.

Fusion reactions produce incredibly energetic particles: neutrons, ions, and photons (or gammas). In magnetic confinement fusion, the charged fast ions are trapped by the magnetic field and deposit their energy in the background plasma from which they were born. Neutral neutrons and gammas, on the other hand, fly out of the device and can be detected; in fact, the ultimate goal is for their energy to be captured to produce electricity one day.

We can measure these energetic particles in time, space, and energy to diagnose the fusion performance of the plasma. In SPARC, these detectors will directly measure fusion power and show energy breakeven for the first time in magnetic fusion! We will also investigate novel physics in “burning plasmas,” with dominant self-heating from the confined ions.

Structural design: simulate how neutrons and gammas interact with surrounding materials and detectors with Monte Carlo codes like OpenMC

- We model how confined fast ions can drive magnetohydrodynamic instabilities and their resulting interactions
- We prototype novel nuclear detectors and test them experimentally with neutron, ion, and gamma sources.

Power exhaust for potential future ARC reactor concepts is studied, utilising large-scale complex plasma edge simulations codes. Three ARC designs are investigated in this work: CFS's ARC V2A, as well as Radiative L-mode and Negative-Triangularity ARC concepts. A range of divertor configurations are studied and examined, from standard divertors to advanced divertor magnetic topologies such as the X-point Target and Super-X Divertors. These results of these simulation studies will enable the identification of divertor configurations viable for a realisable fusion reactor.

The power exhaust projected for future tokamak-based fusion reactors presents a major challenge for fusion energy; fusion power plants will need high radiation in the SOL to reduce divertor plasma temperatures/fluxes to levels that project to realistic lifetime PFC erosion rates.

The divertor targets – specifically armoured plates where the plasma makes contact with the vessel components – need to have heat fluxes/temperatures kept below key material limits to avoid damage from erosion/melting (and subsequent core contamination). Failure of the divertor components at the reactor level can be equated to failure of the device as a whole. However, the divertor power exhaust problem remains unsolved in the fusion community.

The divertor design for ARC is still an open question. Both the vacuum vessel and the divertor designs are intended to remain flexible for as long as possible, so that the results of advanced divertors experiments can inform the final design of ARC. Many potential designs could be considered for an ARC-class device, including different operational modes (H-, I- and L-mode) as well as negative triangularity concepts. Divertor simulations provide the ability to explore different divertor designs and identify the requirements for successful power exhaust in a realisable ARC fusion reactor.

Structure:

```
// Deuterium Fuel Tank
translate([-1.0, 0, 1.0])
    cylinder(h = 0.6, r = 0.12);

// Injection Valve
translate([-0.75, 0, 1.3])
    rotate([0, 90, 0])
        cylinder(h = 0.15, r = 0.02);

// Fuel Conduit
translate([-0.75, 0, 1.3])
    rotate([0, 90, 0])
        cylinder(h = 0.5, r = 0.01);
```

The Tritium breeding and monitoring in molten salt for fusion energy applications: The challenging environment of a fusion reactor requires integrated modeling and experiments to validate conceptual designs, such as molten salt Liquid Immersion Blankets (LIBs). To date, no one has demonstrated tritium breeding in molten salts with a fusion neutron source at scales relevant for commercial fusion energy.

The ambitious goal of LIBRA is to achieve a tritium breeding ratio greater than one, demonstrating the capability of LIB to breed enough tritium, and de-risking the path to a fusion power plant. Additional research objectives include:

- characterizing thermophysical properties of molten salts;
- characterizing fusion neutron sources, and detectors;
- studying tritium transport in molten salt;
- developing and testing tritium accountancy systems;
- establishing manufacturing capabilities for fusion-relevant materials;

investigating safety and handling techniques for hazardous materials.

We can measure neutrons, ions, and photons (or gammas) in time, space, and energy to diagnose the fusion performance of the plasma.

Structure:

Fuel tank

```
bpy.ops.mesh.primitive_cylinder_add(radius=0.12, depth=0.6,  
location=(-1.0, 0, 1.3))
```

Injector valve

```
bpy.ops.mesh.primitive_cylinder_add(radius=0.02, depth=0.15,  
location=(-0.75, 0, 1.3), rotation=(0, math.pi/2, 0))
```

Fuel conduit

```
bpy.ops.mesh.primitive_cylinder_add(radius=0.01, depth=0.5,  
location=(-0.5, 0, 1.3), rotation=(0, math.pi/2, 0))
```

References: Fusion reactions produce incredibly energetic particles: neutrons, ions, and photons (or gammas). In magnetic confinement fusion, the charged fast ions are trapped by the magnetic field and deposit their energy in the background plasma from which they were born. Neutral neutrons and gammas, on the other hand, fly out of the device and can be detected; in fact, the ultimate goal is for their energy to be captured to produce electricity one day.

We can measure these energetic particles in time, space, and energy to diagnose the fusion performance of the plasma. In SPARC, these detectors will directly measure fusion power and show energy breakeven for the first time in magnetic fusion! We will also investigate novel physics in "burning plasmas," with dominant self-heating from the confined ions.

- We simulate how neutrons and gammas interact with surrounding materials and detectors with Monte Carlo codes like OpenMC
- We model how confined fast ions can drive magnetohydrodynamic instabilities and their resulting interactions
- We prototype novel nuclear detectors and test them experimentally with neutron, ion, and gamma sources

Materials project:

<https://next-gen.materialsproject.org/materials/mp-757388>